

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут ім. І. Сікорського»
Кафедра інженерії програмного забезпечення в енергетиці

Практична робота № 2

з курсу: «Компоненти програмної інженерії. Частина 4. Якість та тестування
програмного забезпечення»

Тема: «Розробка тест-плану для програмної системи»

ТЕСТ-ПЛАН

проєкту «Мультиагентна програмна система моделювання та координації групи
безпілотних літальних апаратів на основі Mesa»
Ідентифікатор: UAVS-MTP-1.0

Виконали:
студенти 3-го курсу, групи ТВ-43
Степаненко Назар Юрійович
Аніщенко Артем Олександрович
Перевірив:
доц., к.т.н. Варава І.А.

Київ 2026

ЗМІСТ

1. Ідентифікатор тест-плану (Test Plan Identifier)	3
2. Вступ (Introduction)	3
3. Об'єкти тестування (Test Items)	5
4. Функції, що тестуються (Features To Be Tested).....	6
5. Функції, що не тестуються (Features Not To Be Tested)	8
6. Підхід до тестування (Approach)	8
7. Критерії проходження тестів (Item Pass/Fail Criteria)	12
8. Критерії призупинення та відновлення тестування (Suspension Criteria and Resumption Requirements).....	13
9. Результати тестування (Test Deliverables)	13
10. Тестові завдання (Test Tasks)	14
11. Вимоги до середовища (Environmental Needs).....	15
12. Відповідальність (Responsibilities)	15
13. Кадрове забезпечення та навчання (Staffing and Training Needs)	17
14. Розклад (Schedule)	17
15. Ризики та план дій (Risks and Contingencies)	18
16. Затвердження (Approvals).....	19
Додаток А. Атрибути баг-репорту	19
Додаток Б. Шкала серйозності дефектів.....	20

1. Ідентифікатор тест-плану (Test Plan Identifier)

Атрибут	Значення
Ідентифікатор	UAVS-MTP-1.0
Коротка назва	Головний тест-план проєкту UAV Swarm Mesa
Рівень плану	Головний (Master Test Plan). Охоплює всі рівні тестування — від модульного до приймального. Окремі плани рівнів не створюються: команда складається з двох осіб
Об'єкт	Програмна система «Мультиагентна програмна система моделювання та координації групи безпілотних літальних апаратів на основі Mesa», інкременти 0.1–1.0
Версія і дата	1.0 від 04.10.2026
Автори	Степаненко Н.Ю. (тімлід), Аніщенко А.О. (QA-відповідальний), група ТВ-43
Контакти	Jira-простір UAVS (tv43-uav-swarm.atlassian.net), корпоративна пошта КПІ
Статус	Подано на затвердження

Історія змін

Версія	Дата	Зміни	Автори
1.0	04.10.2026	Початкова версія плану	Степаненко Н.Ю., Аніщенко А.О.

Тест-план зберігається в репозиторії проєкту поряд із кодом і переглядається на плануванні кожного спринту. Уточнення розкладу чи переліку функцій збільшують молодший номер версії (1.1, 1.2), зміна підходу до тестування — старший (2.0).

2. Вступ (Introduction)

2.1. Призначення документа

Документ визначає, що саме, яким чином, якими силами й у які терміни буде протестовано в командному проєкті з моделювання рою безпілотних літальних апаратів (БПЛА). Це план найвищого рівня: на нього спираються специфікації тестів, тестові завдання в Jira та підсумковий звіт про тестування.

План складено за шаблоном IEEE 829-1998 з урахуванням двох особливостей проєкту. По-перше, об'єктом тестування є агентна імітаційна модель, поведінка якої залежить від генератора випадкових чисел. По-друге, окремої групи тестувальників немає: тестують ті самі двоє студентів, які пишуть код.

2.2. Про проєкт

Агентна модель рою БПЛА на Python/Mesa: середовище з перешкодами та забороненими зонами, агенти-дрони з обмеженим зарядом і сенсорами, децентралізований розподіл задач, уникнення зіткнень, сценарії пошуку та патрулювання, візуалізація Solara і збір метрик для порівняння стратегій координації.

Система розробляється мовою Python 3.12 на фреймворку Mesa 3 з використанням NumPy, Solara, pytest і GitHub Actions. Робота ведеться за Scrum сімома двотижневими спрінтами (14.09–20.12.2026). Беклог налічує 8 епіків і 22 історії користувача загальною оцінкою 90 story points.

2.3. Обсяг і обмеження

- **Обсяг.** Усі функції, заплановані в беклозі до релізу 1.0, та нефункціональні характеристики якості за ISO/IEC 25010: продуктивність, надійність, супроводжуваність, переносимість, зручність використання.
- **Ресурси.** Двоє виконавців, кожен поєднує ролі розробника й тестувальника. Бюджету, виділеного тестового стенда та платних інструментів немає.
- **Термін.** Дата релізу фіксована — 20.12.2026. Додатковий час на тестування можна отримати лише зменшенням обсягу функцій.
- **Зв'язок з іншими перевірками.** Тестування доповнює статичні методи: рецензування коду в pull request, аналіз вимог перед плануванням спринту та автоматичний статичний аналіз (ruff).
- **Керування змінами.** Вимоги змінюються лише через Jira за згодою Product Owner. Зміна критеріїв приймання історії тягне за собою перегляд її тестів.

2.4. Посилання

1. IEEE Std 829-1998. IEEE Standard for Software Test Documentation — структура тест-плану.
2. Беклог продукту: Jira-простір UAVS, tv43-uav-swarm.atlassian.net (епіки UAVS-5...UAVS-12, історії UAVS-13...UAVS-34).
3. Протокол практичної роботи № 1 «Система управління розробкою як інструмент управління якістю» — склад команди, ролі, план спринтів.
4. ISO/IEC 25010. System and software quality models — характеристики якості продукту.
5. ISO/IEC/IEEE 12207. Software life cycle processes — процеси верифікації та валідації.

6. Лекції 1–5 курсу «Якість та тестування програмного забезпечення»: ролі тестувальника, моделі й метрики якості, верифікація та валідація, відстеження дефектів.
7. Документація Mesa 3 (mesa.readthedocs.io) та PEP 8 — Style Guide for Python Code.

2.5. Терміни

Термін	Значення в цьому плані
Помилка (error)	Дія людини, що приводить до некоректного результату
Дефект (defect, bug)	Недолік у компоненті або системі, який може спричинити збій чи відмову
Збій	Самоусувна або одноразова відмова, яку усуває незначне втручання
Відмова (failure)	Порушення працездатного стану системи, помітне користувачеві
Верифікація	Перевірка, чи робимо ми продукт правильно: відповідність коду вимогам і специфікаціям. Виконують розробники
Валідація	Перевірка, чи робимо ми правильний продукт: чи задовольняє система потреби замовника. Виконують тестувальники
Seed	Початкове значення генератора випадкових чисел моделі. Однакові сценарій і seed мають давати однаковий перебіг симуляції
Інваріант	Властивість рою, яка має виконуватися на кожному кроці симуляції, наприклад «відстань між будь-якими двома дронами не менша за <code>safe_distance</code> »

3. Об'єкти тестування (Test Items)

Тестуються програмні компоненти пакета `uav_swarm` та документація, що постачається разом із системою. Вимогами до кожного об'єкта є історія користувача відповідного епіка з їхніми критеріями приймання.

Об'єкт тестування	Що входить	Епік	Інкремент
Модель середовища польоту	Неперервний 2D-простір, перешкоди, заборонені для польоту зони, база зарядки, цілі пошуку	UAVS-6	0.1
Агент БПЛА	Рух з обмеженою швидкістю, витрата заряду, повернення на базу, сенсор виявлення цілей	UAVS-7	0.1–0.2
Комунікація та координація рою	Обмін повідомленнями в радіусі зв'язку, аукціонний розподіл цілей, уникнення зіткнень, строй	UAVS-8	0.3–0.4
Сценарії місій	Сценарій пошуку цілей на площі, опис сценаріїв у YAML та його валідація	UAVS-9	0.4
Візуалізація	Карта з дронами, перешкодами й цілями в браузері (SolaraViz), керування симуляцією, слайдери параметрів	UAVS-10	0.5
Збір даних та аналітика	Метрики місії на кожному кроці, експорт CSV, серії експериментів <code>batch run</code>	UAVS-11	0.5
Інфраструктура проєкту	Структура пакета, <code>pyproject.toml</code> із залежностями, конвеєр CI	UAVS-5	0.1

Об'єкт тестування	Що входить	Епік	Інкремент
Документація користувача	README з інструкцією встановлення і запуску	UAVS-5	1.0

Інкремент — це стан гілки main наприкінці спринту, позначений тегом Git: 0.1 після спринту 2, далі по одному інкременту на спринт, 1.0 після спринту 7.

Умови початку тестування об'єкта

- код історії подано як pull request, а модульні тести автора проходять;
- конвеєр CI на гілці зелений: ruff без зауважень, pytest без падінь;
- система встановлюється в чисте віртуальне середовище командою з README;
- історія в Jira перенесена в колонку Testing і містить критерії приймання.

На момент складання плану зареєстрованих дефектів і запитів на зміни немає: розробка коду починається у спринті 2.

Що не є об'єктом тестування

Сторонні бібліотеки (Mesa, NumPy, Solara) та сервіси (Jira, GitHub) самі по собі не тестуються: перевіряється лише те, як їх використовує система. Докладніше — у розділі 5.

4. Функції, що тестуються (Features To Be Tested)

Функції описано з погляду користувача системи — дослідника, який моделює місію рою. Рівень ризику: Н — високий, М — середній, L — низький. Він визначає глибину тестування та порядок скорочення обсягу, якщо забракне часу. Кожна група функцій має свою специфікацію тестів (TDS).

Таблиця 4.1 — Функціональні можливості

ID	Функція з погляду користувача	Історія, TDS	Ризик	Чому такий рівень
F-01	Задати розміри карти, перешкоди й заборонені для польоту зони	UAVS-16 TDS-ENV	Н	На геометрії середовища базуються рух, пошук і обхід перешкод; помилка спотворює всі результати
F-02	Розмістити на карті базу зарядки та цілі пошуку	UAVS-17 TDS-ENV	М	Розміщення цілей залежить від seed; помилку видно на візуалізації, цілі можна задати вручну
F-03	Спрямувати БПЛА до точки з обмеженою швидкістю, без входу в перешкоди	UAVS-18 TDS-AGENT	Н	Базова поведінка агента; дефект робить непридатними всі сценарії

ID	Функція з погляду користувача	Історія, TDS	Ризик	Чому такий рівень
F-04	Враховувати витрату заряду й автоматичне повернення на базу	UAVS-19 TDS-AGENT	Н	Логіка станів і порогових значень; помилка призводить до «втрати» дронів і хибних метрик
F-05	Виявляти цілі в радіусі сенсора	UAVS-20 TDS-AGENT	М	Проста геометрична перевірка, проте чутлива до значень на межі радіуса
F-06	Обмінюватися повідомленнями лише в радіусі зв'язку, з можливою втратою пакетів	UAVS-21 TDS-COORD	Н	Випадкова втрата пакетів і доставка із затримкою на крок; дефекти важко відтворити
F-07	Розподіляти цілі між БПЛА аукціоном без дублювання	UAVS-22 TDS-COORD	Н	Найскладніший алгоритм проекту (8 SP); помилка порушує головну властивість координації
F-08	Уникати зіткнень між БПЛА	UAVS-23 TDS-COORD	Н	Властивість безпеки, що має виконуватися на кожному кроці для будь-якого seed
F-09	Вести групу у строю (лінія, клин)	UAVS-24 TDS-COORD	L	Допоміжна функція з низьким пріоритетом у беклозі; на сценарій пошуку не впливає
F-10	Запустити сценарій пошуку цілей і отримати відсоток покриття території	UAVS-25 TDS-SCEN	Н	Наскрізний сценарій, що поєднує всі модулі; саме його демонструють на захисті
F-11	Описати сценарій у YAML-файлі й отримати зрозуміле повідомлення про помилку в ньому	UAVS-26 TDS-SCEN	М	Багато класів некоректних даних; відмова тут не має зупиняти програму аварійно
F-12	Бачити карту, дрони, перешкоди й цілі в браузері; керувати симуляцією (старт, пауза, крок)	UAVS-27 TDS-UI	М	Перевіряється лише вручну; на результати розрахунків не впливає
F-13	Змінювати параметри слайдерами й бачити графік знайдених цілей	UAVS-28 TDS-UI	L	Стосується зручності; логіка перезапуску моделі проста
F-14	Збирати метрики місії на кожному кроці та експортувати їх у CSV	UAVS-29 TDS-DATA	М	Від метрик залежать висновки дослідження, але самі розрахунки нескладні
F-15	Запускати серії експериментів і порівнювати стратегії координації	UAVS-30 TDS-DATA	М	Помилка в агрегації результатів дає хибне порівняння стратегій
F-16	Відтворити перебіг симуляції: однакові сценарій і seed дають однаковий результат	UAVS-17, UAVS-32 TDS-SCEN	Н	Наскрізна властивість; без неї автотести нестабільні, а дефекти не відтворюються

Таблиця 4.2 — Нефункціональні характеристики

ID	Характеристика (ISO/IEC 25010)	Що перевіряється	Ризик
N-01	Продуктивність	Сценарій пошуку з 50 БПЛА на 1000 кроків без візуалізації виконується не довше 60 с на ноутбучі розробника	М

ID	Характеристика (ISO/IEC 25010)	Що перевіряється	Ризик
N-02	Надійність	Некоректний YAML чи параметр не спричиняє аварійного завершення; прогін на 10 000 кроків проходить без відмов	M
N-03	Супроводжуваність	Покриття коду тестами не менше 80 %; ruff без зауважень; цикломатична складність функцій V(G) не більша за 10	M
N-04	Переносимість	Система встановлюється й проходить тести на Windows 11 та Ubuntu (середовище CI) з Python 3.12	L
N-05	Зручність використання	Новий користувач запускає симуляцію за інструкцією README без сторонньої допомоги	L

5. Функції, що не тестуються (Features Not To Be Tested)

Що не тестується	Причина
Внутрішня коректність Mesa, NumPy, Solara	Сторонні бібліотеки зі стабільними версіями, які тестують їхні розробники. Версії зафіксовано в pyproject.toml
Робота з реальними БПЛА, польотними контролерами, радіоканалом	Система є імітаційною моделлю; апаратна частина поза межами проєкту
Тривимірний політ, вітер та інші погодні умови	Не входять у реліз 1.0: модель працює у двовимірному просторі
Сценарій патрулювання периметра	Згаданий в описі епіка UAVS-9, але історії на нього в беклозі немає — у реліз 1.0 не входить
Захищеність (автентифікація, розмежування доступу)	Локальний однокористувацький застосунок без мережевого доступу ззовні; низький ризик
Рій понад 200 БПЛА	Поза заявленими межами застосування; перевіряється лише конфігурація N-01
macOS та браузері, крім Chrome і Edge	У команди немає такого обладнання; низький ризик, бо код не залежить від платформи
Доступність і локалізація інтерфейсу	Вимог до них у беклозі немає
Функції ще не завершених історій	Перевіряються у спрінті, на який заплановані (розділ 14). Повідомлення про «дефект» у нереалізованій функції відхиляється

6. Підхід до тестування (Approach)

6.1. Рівні тестування

Рівень	Що перевіряється	Хто виконує	Коли
Модульне	Окремі класи й функції: геометрія простору, крок руху, витрата заряду, виявлення цілі, розрахунок ставки	Розробник історії	Разом із кодом, до pull request
Інтеграційне	Взаємодія агентів через модель: обмін повідомленнями, аукціон, уникнення зіткнень; інваріанти рою	Тестувальник історії (другий учасник)	Після злиття історій, від яких залежить функція

Рівень	Що перевіряється	Хто виконує	Коли
Системне	Наскрізні сценарії з YAML-конфігурації, нефункціональні характеристики, інтерфейс	Тестувальник історії; інтерфейс — вручну	Наприкінці спринту, на зібраному інкременті
Приймальне	Відповідність критеріям приймання історій і потребам замовника	Product Owner; викладач на демонстрації	Огляд кожного спринту; підсумкове — у спринті 7

6.2. Види тестування

- **Функціональне** — перевірка кожного критерію приймання історії.
- **Регресійне** — автоматичний повторний запуск усіх тестів після кожної зміни (п. 6.9).
- **Димове (smoke)** — модель створюється із типового сценарію й виконує 10 кроків без винятків; запускається першим у CI.
- **Продуктивності** — вимірювання часу виконання конфігурації N-01.
- **Надійності** — некоректні вхідні дані та тривалий прогін на 10 000 кроків.
- **Статичне** — рецензування вимог і коду, аналіз ruff без запуску програми.

Функції координації F-06, F-07 і F-08 на інтеграційному рівні перевіряються лише разом, на зібраній моделі: інваріанти рою виникають із їхньої взаємодії, і тест однієї функції окремо їх не покаже. Візуалізація тестується лише на повному інкременті.

6.3. Верифікація та валідація

Верифікацію виконує автор коду: модульні тести, статичний аналіз і перевірка відповідності критеріям приймання до подання pull request. Валідацію виконує другий учасник у ролі тестувальника: він запускає систему на сценаріях і оцінює, чи отримує дослідник те, що йому потрібно. Остаточну валідацію проводить викладач як замовник на демонстрації інкремента.

6.4. Техніки проєктування тестів

Техніка	Де застосовується в проєкті
Класи еквівалентності та граничні значення	Заряд відносно порога повернення; відстань до цілі відносно <code>sensor_range</code> ; відстань між дронами відносно <code>comm_range</code> і <code>safe_distance</code> ; швидкість відносно <code>max_speed</code> ; точка на межі перешкоди
Таблиці рішень	Розрахунок ставки в аукціоні (відстань і рівень заряду); валідація полів YAML
Переходи станів	Життєвий цикл БПЛА: політ → повернення на базу → зарядка → політ

Техніка	Де застосовується в проєкті
Перевірка інваріантів	«Ціль закріплена не більше ніж за одним дроном», «відстань між дронами не менша за <code>safe_distance</code> », «дрон не входить у перешкоду» — на кожному кроці для кожного seed із набору
Базисні шляхи (метрика МакКейба)	Мінімальна кількість тестів функції дорівнює її цикломатичному числу $V(G)$; застосовується до функцій з розгалуженою логікою (аукціон, вибір стану)
Статистична перевірка	Частка втрачених пакетів відповідає заданій імовірності в межах допуску; порівняння стратегій на серії seed
Дослідницьке тестування	Інтерфейс візуалізації

Головна складність проєкту — тестовий оракул. Для рою з десятків агентів неможливо заздалегідь обчислити очікувані координати кожного дрона. Тому точні очікувані значення задаються лише для малих сценаріїв (1–3 дрони, результат обчислено вручну), а для великих перевіряються інваріанти та межі метрик. Усі стохастичні тести виконуються на фіксованому наборі з 20 seed (0–19).

6.5. Інструменти

Інструмент	Призначення	Навчання
pytest	Модульні, інтеграційні та системні автотести	Так: фікстури, параметризація, маркери
pytest-cov	Вимірювання покриття коду	Ні
ruff	Стиль PEP 8, типові помилки, цикломатична складність (правило C901)	Ні
GitHub Actions	Запуск перевірок на кожен push і pull request	Так, базове
Jira, простір UAVS	Беклог, тестові завдання, баг-репорти, дошка	Ні (опановано в ПР № 1)
Git, GitHub	Версії коду, тестів і тестових даних; рецензування	Ні
Chrome, Edge	Ручне тестування інтерфейсу Solara	Ні

6.6. Метрики

Метрика	Рівень збору	Цільове значення
Покриття коду тестами (рядки)	Модульний, інтеграційний	Не менше 80 %
Частка пройдених тестів	Усі рівні	100 % автотестів у main; не менше 95 % усіх тест-кейсів на реліз
Покриття вимог: історії, де кожен критерій приймання має тест	Системний	100 % історій спринту
Відкриті дефекти за серйозністю S1–S7	Проєкт	S1–S2: жодного на кінець спринту
Щільність дефектів (на 1000 рядків коду, SLOC)	Проєкт	Відстежується, порога немає
Цикломатична складність МакКейба $V(G)$	Функція	Не більше 10; інакше рефакторинг

Метрика	Рівень збору	Цільове значення
PRAT — якість тестування елементів беклогу	Спрінт	Розраховується на ретроспективі, порівнюється між спрінтами
Кроки симуляції між відмовами (аналог MTBF)	Системний	Жодної відмови на 10 000 кроків

6.7. Керування конфігурацією

Код, тести, тестові сценарії та цей план зберігаються в одному Git-репозиторії, тож версія тестів завжди відповідає версії коду. Кожна історія розробляється в окремій гілці; у main вона потрапляє через pull request після рецензії другим учасником і зеленого CI. Назва гілки й повідомлення комітів містять ключ задачі Jira. Інкремент наприкінці спринту позначається тегом. Версії залежностей зафіксовано, оновлюються вони окремою задачею.

6.8. Конфігурації, що тестуються

№	Операційна система	Python і залежності	Що запускається
1	Ubuntu (GitHub Actions, ubuntu-latest)	Python 3.12, версії з pyproject.toml	Усі автотести на кожен push
2	Windows 11 (ноутбуки розробників)	Python 3.12, версії з pyproject.toml	Усі автотести перед pull request; ручні тести інтерфейсу
3	Windows 11, браузер Chrome та Edge	Solara у складі системи	Чек-лист інтерфейсу (F-12, F-13)

6.9. Правила регресійного тестування

- усі автотести запускаються в CI на кожен push; злиття в main із червоним CI заборонене;
- на кожен виправлений дефект S1–S3 додається автотест, що його відтворює, і лише після цього дефект закривається;
- після виправлення дефекту S1–S2 заново проходять усі тести ураженого модуля та димовий тест;
- перед оглядом спринту вручну проходиться чек-лист інтерфейсу;
- у спринті 7 виконується повна регресія на конфігураціях 1–3.

6.10. Вимоги, які неможливо протестувати

Перед плануванням спринту історії переглядає учасник у ролі тест-аналітика. Якщо критерій приймання не можна перевірити (немає числового

порога, незрозумілий очікуваний результат), історія з коментарем повертається Product Owner і до спринту не береться, доки критерій не уточнено.

6.11. Організація роботи та обмеження

Тестові завдання оцінюються на плануванні спринту разом із розробкою. Стан тестування обговорюється на щотижневій зустрічі команди; на огляді спринту демонструється інкремент, на ретроспективі розглядаються метрики з п. 6.6. Основні обмеження: двоє виконавців, навчальне навантаження з інших дисциплін, фіксована дата релізу.

6.12. Керування дефектами

Дефекти реєструються в Jira як робочі елементи типу Bug з обов'язковими атрибутами баг-репорту (додаток А); цей тип буде додано до простору UAVS у завданні T2. Серйозність за шкалою S1–S7 (додаток Б) встановлює тестувальник, який знайшов дефект. Пріоритет (High, Medium, Low) визначає Product Owner; дефекти усуваються в порядку High → Medium → Low.

Стан дефекту	Колонка дошки Jira	Хто переводить
Новий (New)	To Do, без виконавця	Тестувальник, який знайшов дефект
Призначений (Assigned)	To Do, з виконавцем	Product Owner
Відкритий, у роботі (Open)	In Progress	Розробник
Виправлений (Fixed)	In Review	Розробник, після pull request
На перевірці (Verify)	Testing	Тестувальник
Закритий (Closed)	Done	Тестувальник, після успішної перевірки
Перевідкритий (Reopened)	To Do	Тестувальник, якщо дефект відтворюється
Відхилений або відкладений (Rejected, Deferred)	Done з міткою rejected або беклог з міткою deferred	Product Owner

7. Критерії проходження тестів (Item Pass/Fail Criteria)

Рівень	Критерій «пройдено»
Тест-кейс	Фактичний результат збігається з очікуваним. Для стохастичного тесту — інваріант виконується на всіх 20 seed набору
Модуль	Усі модульні тести проходять; покриття модуля не менше 80 %; ruff без зауважень; V(G) кожної функції не більше 10
Історія користувача	Кожен критерій приймання має щонайменше один тест, і всі вони проходять; pull request схвалив другий учасник; за історією немає відкритих дефектів S1–S3
Спринт	Виконано всі тестові завдання спринту; CI на main зелений; немає відкритих дефектів S1–S2

Рівень	Критерій «пройдено»
Реліз 1.0	Виконано критерії всіх спринтів; пройдено 100 % тестів функцій із ризиком Н і не менше 95 % усіх тест-кейсів; жодного відкритого дефекту S1–S2 і не більше двох S3 з описаним обхідним шляхом; покриття не менше 80 %; підготовлено підсумковий звіт

Об'єкт вважається таким, що не пройшов тестування, якщо не виконано хоча б один критерій його рівня. Дефекти S4–S7 не блокують проходження: вони фіксуються в Jira й переходять у беклог з пріоритетом, який визначає Product Owner. Кількість знайдених дефектів не дорівнює кількості наявних: надійність оцінюється за частотою відмов у прогонах, а не за числом баг-репортів.

8. Критерії призупинення та відновлення тестування (Suspension Criteria and Resumption Requirements)

Тестування призупиняється, якщо	Відновлюється, коли
Димовий тест не проходить: модель не створюється або не виконує крок	Збірку виправлено, димовий тест проходить; усі тести запускаються заново
Відкрито дефект S1 (Blocker) у модулі, від якого залежать інші (середовище, базовий агент)	Дефект виправлено й перевірено; заново проходять тести модуля та залежних від нього функцій
В одному прогоні падає понад 30 % тестів — подальші результати будуть «примарними» наслідками першої помилки	Знайдено й усунуто першопричину; повний прогін повторено
Середовище недоступне: не працює GitHub Actions або нова версія залежності ламає встановлення	Сервіс відновлено чи версію залежності зафіксовано; тести запускаються локально на конфігурації 2
Критерії приймання історії змінено під час спринту	Product Owner затвердив нові критерії, тести історії оновлено

Рішення про призупинення приймає тімлід за поданням тестувальника; причину записують у коментарі до задачі Jira. Доки діє призупинення, тестувальник готує тести для наступних історій.

9. Результати тестування (Test Deliverables)

Документ або артефакт	Форма і місце зберігання	Коли готовий
Тест-план (цей документ)	DOCX і PDF у репозиторії	04.10.2026
Специфікації тестів TDS-ENV, TDS-AGENT, TDS-COORD, TDS-SCEN, TDS-UI, TDS-DATA	Чек-листи умов і тестових даних у каталозі docs/testing	На початку спринту, де тестується група
Автоматизовані тест-кейси	Код pytest у каталозі tests	Разом з історією
Ручні тест-кейси інтерфейсу	Чек-лист TDS-UI	Спринт 6
Тестові дані	YAML-сценарії та список seed у tests/data	Разом із тестами

Документ або артефакт	Форма і місце зберігання	Коли готовий
Матриця трасування «історія → критерій приймання → тест»	Таблиця в репозиторії	Оновлюється кожного спринту
Журнали тестування	Журнали запусків GitHub Actions, звіт покриття	Автоматично, на кожен push
Звіти про дефекти	Робочі елементи Bug у Jira	У міру виявлення
Звіт про тестування спринту	Короткий підсумок метрик на огляді спринту	Кінець кожного спринту
Підсумковий звіт про тестування	Документ у репозиторії (UAVS-34)	Спринт 7, до 18.12.2026

Саме програмне забезпечення до результатів тестування не належить: воно є об'єктом тестування (розділ 3) і постачається розробкою.

10. Тестові завдання (Test Tasks)

№	Завдання	Залежить від	Спринт	Задача Jira
T1	Розробити, відрецензувати й затвердити тест-план	Беклог і план спринтів (ПР № 1)	2	UAVS-31, UAVS-42, UAVS-43
T2	Додати в простір UAVS тип Bug і шаблон баг-репорту	T1	2	UAVS-31
T3	Налаштувати pytest, pytest-cov і ruff у CI; перевірити CI на тестовому pull request	Каркас проєкту (UAVS-14)	2	UAVS-15, UAVS-40, UAVS-41
T4	TDS-ENV, TDS-AGENT: тести середовища польоту та руху БПЛА	T3; UAVS-16...UAVS-18	2	створюється
T5	TDS-AGENT: тести заряду, повернення на базу, сенсора	T4; UAVS-19, UAVS-20	3	UAVS-32
T6	TDS-COORD: тести обміну повідомленнями та аукціону	T5; UAVS-21, UAVS-22	4	створюється
T7	TDS-COORD: інваріанти рою (зіткнення, дублювання цілей), строй	T6; UAVS-23, UAVS-24	5	UAVS-33
T8	TDS-SCEN: наскрізний сценарій пошуку, YAML, відтворюваність за seed	T7; UAVS-25, UAVS-26	5	UAVS-33
T9	TDS-UI та TDS-DATA: інтерфейс, метрики, серії експериментів	T8; UAVS-27...UAVS-30	6	створюється
T10	Тести продуктивності й надійності (N-01, N-02)	T8	6	створюється
T11	Повна регресія, приймальне тестування, підсумковий звіт	T1–T10	7	UAVS-34
T12	Реєстрація та перевірка виправлення дефектів, збір метрик	—	2–7	Bug у Jira

Завдання з позначкою «створюється» додаються в Jira як підзадачі відповідних історій на плануванні свого спринту. Навички, потрібні для завдань, і навчання описано в розділі 13.

Система постачається інкрементами, тож у кожному спрінті тестуються лише завершені історії. Функції наступних спрінтів (розділ 14) у поточному інкременті не перевіряються, і повідомлення про дефекти в них не приймаються.

11. Вимоги до середовища (Environmental Needs)

- **Апаратне забезпечення.** Ноутбуки учасників (x86-64, від 8 ГБ оперативної пам'яті). Спеціального обладнання, симуляторів чи реальних БПЛА не потрібно: система сама є імітаційною моделлю.
- **Програмне забезпечення.** Python 3.12; Mesa 3, NumPy, Solara, pytest, pytest-cov, ruff у версіях, зафіксованих у `pyproject.toml`. Встановлення в окреме віртуальне середовище.
- **Середовище CI.** Безкоштовний виконавець GitHub Actions `ubuntu-latest`.
- **Тестові дані.** Створюються командою і зберігаються в `tests/data`: мала карта без перешкод для ручного розрахунку; карта з перешкодами й забороненою зоною; граничні сценарії (один дрон, жодної цілі, ціль на межі радіуса сенсора, заряд на порозі повернення); некоректні YAML-файли; сценарій N-01 на 50 БПЛА. Реальні дані не потрібні.
- **Мережа.** Доступ до інтернету потрібен лише для встановлення залежностей і роботи з GitHub та Jira. Самі тести мережі не потребують; інтерфейс Solara відкривається на `localhost`.
- **Обмеження використання.** Тестування продуктивності виконується на ноутбучі без інших навантажених процесів; результат записується разом з моделлю процесора.

12. Відповідальність (Responsibilities)

У команді двоє учасників, тому ролі тестувальника, відомі з курсу (Test Manager, Test Analyst, Test Designer, Test Executor), розподілено між ними. Діє правило перехресного тестування: функцію тестує не той, хто її розробив. Так кожен студент працює і розробником, і тестувальником, а автор не перевіряє власний код наодинці.

Обов'язок	Відповідальний
Test Manager: планування тестування, контроль термінів і покриття вимог, рішення «випускати чи ні» для інкремента	Степаненко Н.Ю. (тімлід)

Обов'язок	Відповідальний
Ведення тест-плану та шаблону баг-репорту	Аніщенко А.О. (QA-відповідальний); рецензує Степаненко Н.Ю.
Test Analyst: перевірка повноти й однозначності критеріїв приймання перед спрінтом	Тестувальник історії (таблиця 12.1)
Test Designer, Test Executor: проектування й виконання інтеграційних і системних тестів, реєстрація дефектів	Тестувальник історії
Модульні тести та виправлення дефектів	Розробник історії
Передача об'єкта на тестування: pull request з історією та модульними тестами	Розробник історії
Розв'язання конфліктів у розкладі тестування	Степаненко Н.Ю. (тімлід)
Оцінка ризиків, вибір функцій для тестування	Обидва учасники на плануванні; остаточно — Product Owner (Степаненко Н.Ю.)
Серйозність дефекту	Тестувальник, який його знайшов
Пріоритет дефекту	Product Owner (Степаненко Н.Ю.)
Працездатність CI і тестового середовища	Степаненко Н.Ю. (автор UAVS-15)
Навчання роботі з інструментами	Кожен готує коротке пояснення за своєю темою (розділ 13)
Рішення в ситуаціях, не описаних планом	Тімлід; за потреби — викладач
Приймання результатів	Варава І.А. (викладач, замовник)

Таблиця 12.1 — Перехресне тестування функцій

Функція	Історія	Спрінт	Розробляє, пише модульні тести	Тестує (інтеграційні, системні)
F-01	UAVS-16	2	Степаненко Н.Ю.	Аніщенко А.О.
F-02	UAVS-17	2	Аніщенко А.О.	Степаненко Н.Ю.
F-03	UAVS-18	2	Аніщенко А.О.	Степаненко Н.Ю.
F-04	UAVS-19	3	Степаненко Н.Ю.	Аніщенко А.О.
F-05	UAVS-20	3	Аніщенко А.О.	Степаненко Н.Ю.
F-06	UAVS-21	4	Степаненко Н.Ю.	Аніщенко А.О.
F-07	UAVS-22	4	Аніщенко А.О.	Степаненко Н.Ю.
F-08	UAVS-23	5	Степаненко Н.Ю.	Аніщенко А.О.
F-09	UAVS-24	5	Аніщенко А.О.	Степаненко Н.Ю.
F-10	UAVS-25	5	Аніщенко А.О.	Степаненко Н.Ю.
F-11	UAVS-26	5	Степаненко Н.Ю.	Аніщенко А.О.
F-12	UAVS-27	6	Степаненко Н.Ю.	Аніщенко А.О.
F-13	UAVS-28	6	Аніщенко А.О.	Степаненко Н.Ю.
F-14	UAVS-29	6	Аніщенко А.О.	Степаненко Н.Ю.
F-15	UAVS-30	6	Степаненко Н.Ю.	Аніщенко А.О.

Навантаження розподілено рівномірно: Степаненко Н.Ю. розробляє 7 функцій і тестує 8, Аніщенко А.О. — розробляє 8 і тестує 7. Наскрізню властивість F-16 та нефункціональні характеристики N-01...N-05 учасники перевіряють разом у спрінтах 5–7.

13. Кадрове забезпечення та навчання (Staffing and Training Needs)

Тестуванням займаються двоє студентів третього курсу; найняти чи залучити додаткових виконавців неможливо. На тестові роботи кожен планує орієнтовно 3–4 години на тиждень, у спринті 7 — більше. Досвіду комерційного тестування в команді немає, тому на початку потрібне навчання.

Тема	Навіщо	Як і коли
Mesa 3: агенти, простір, DataCollector, batch_run, керування seed	Розуміти об'єкт тестування і писати відтворювані тести	Офіційна документація та приклади; спринт 2
pytest: фікстури, параметризація, маркери; pytest-cov	Автоматизація тестів усіх рівнів	Документація; спільна сесія з написання перших тестів; спринт 2
Техніки проектування тестів, метрики, серйозність і життєвий цикл дефекту	Проектувати тести й оформлювати баг-репорти однаково	Лекції курсу; до початку спринту 3
GitHub Actions	Підтримувати CI	Степаненко Н.Ю. показує конфігурацію після UAVS-15
Баг-репорти в Jira	Єдиний процес роботи з дефектами	Аніщенко А.О. показує шаблон після T2

14. Розклад (Schedule)

Спринт	Дати	Мета розробки	Інкр.	Тестові роботи
1	14.09–27.09	Вимоги, Jira, каркас проєкту та CI	—	Вимоги й Jira (ПР № 1). Тестові роботи не виконувалися
2	28.09–11.10	Модель середовища і базовий агент БПЛА	0.1	T1–T4: тест-план, тип Bug, тестова інфраструктура, тести середовища й руху
3	12.10–25.10	Рух, енергія, сенсори; unit-тести агентів	0.2	T5: тести заряду й сенсора БПЛА
4	26.10–08.11	Комунікація і розподіл задач	0.3	T6: тести комунікації та аукціону
5	09.11–22.11	Сценарії місій і уникнення зіткнень	0.4	T7, T8: інваріанти рою, наскрізний сценарій
6	23.11–06.12	Візуалізація, метрики, batch-експерименти	0.5	T9, T10: інтерфейс, аналітика, продуктивність і надійність
7	07.12–20.12	Стабілізація, регресійне тестування, звіт і захист	1.0	T11: повна регресія, приймальне тестування, підсумковий звіт

Віхи тестування

Віха	Подія	Дата
M1	Тест-план затверджено	04.10.2026
M2	Тестова інфраструктура працює: CI запускає тести й вимірює покриття	11.10.2026
M3	Модульні тести середовища й агента готові, покриття не менше 80 %	25.10.2026
M4	Інтеграційні тести координації рою проходять	22.11.2026

Віха	Подія	Дата
M5	Системні тести сценарію пошуку проходять	22.11.2026
M6	Перевірено інтерфейс, аналітику, продуктивність	06.12.2026
M7	Повна регресія завершена, підсумковий звіт готовий	18.12.2026
M8	Реліз 1.0, демонстрація викладачеві	20.12.2026

Станом на 04.10.2026 історії спринту 1, пов'язані з тест-планом (UAVS-31) і каркасом проєкту (UAVS-14, UAVS-15), ще не закриті й переносяться у спринт 2; тому завдання T1–T3 стоять у розкладі спринту 2.

Правила на випадок відставання

- тестування історії починається не з календарної дати, а з моменту, коли вона потрапляє в колонку Testing; затримка розробки зсуває тестування на той самий строк;
- історія, не протестована до кінця спринту, не вважається завершеною і переноситься в наступний спринт разом зі своїми тестами;
- спринт 7 є резервом для стабілізації: нові функції в ньому не розробляються;
- якщо резерву бракує, обсяг скорочується в порядку зростання ризику — спершу F-09 і F-13 (L), потім функції рівня M; функції рівня H зі складу релізу не вилучаються;
- зменшувати глибину тестування функцій рівня H чи послаблювати критерії розділу 7 заради дати релізу не можна.

15. Ризики та план дій (Risks and Contingencies)

№	Ризик	Рівень	Запобігання	Дії, якщо ризик справдився
R1	Один з двох учасників тимчасово недоступний (хвороба, МКР, дедлайни інших дисциплін)	H	Тести й дані в репозиторії; щотижнева синхронізація; тестові завдання плануються на першу половину спринту	Інший учасник тестує функції рівня H; решта переноситься на наступний спринт
R2	Розробка історії затримується, і на тестування не лишається часу	H	Дати тестування прив'язані до готовності історії; резервний спринт 7	Скорочення обсягу за правилами розділу 14
R3	Нестабільні тести через випадковість симуляції	H	Усі випадкові величини беруться лише з генератора моделі; фіксований набір seed; допуски в статистичних перевірках	Нестабільний тест позначається й розбирається як дефект S3 до кінця спринту
R4	Немає точного очікуваного результату для великого рою	M	Малі сценарії з ручним розрахунком; інваріанти й межі метрик замість точних значень	Функція приймається за інваріантами та демонстрацією викладачеві
R5	Автор тестує власний код і не помічає дефектів	M	Перехресне тестування та рецензія pull request (розділ 12)	Дослідницька сесія тестування удвох перед оглядом спринту

№	Ризик	Рівень	Запобігання	Дії, якщо ризик справдився
R6	Нова версія Mesa чи Solara змінює API	M	Версії залежностей зафіксовано; оновлення окремою задачею з повною регресією	Повернення до попередньої версії залежності
R7	Критерії приймання змінюються під час спринту	M	Зміни лише через Jira за згодою Product Owner; аналіз вимог перед плануванням	Тести оновлюються, історія оцінюється заново; за потреби переноситься
R8	Бракує досвіду з pytest і Mesa	M	Навчання у спринті 2 (розділ 13); спільне написання перших тестів	Спрощення тестів до перевірки критеріїв приймання; консультація викладача
R9	Інтерфейс тестується лише вручну, регресії пропускаються	L	Логіка винесена з інтерфейсу в модель; чек-лист перед кожним оглядом спринту	Дефекти інтерфейсу S4–S7 не блокують реліз
R10	Продуктивність виявиться недостатньою для N-01	M	Перше вимірювання вже у спринті 5, а не наприкінці	Профілювання й оптимізація у спринті 6; інакше поріг N-01 переглядає Product Owner

16. Затвердження (Approvals)

План набуває чинності після погодження командою і затвердження викладачем. Перехід до наступного спринту дозволяє тімлід, якщо виконано критерії розділу 7 для спринту. Реліз 1.0 приймає викладач за підсумковим звітом про тестування та демонстрацією.

Роль	ПІБ	Підпис	Дата
Тімлід, Test Manager, Product Owner	Степаненко Н.Ю.		
QA-відповідальний	Аніщенко А.О.		
Викладач (замовник)	Варава І.А.		

Додаток А. Атрибути баг-репорту

Атрибут	Що вказувати в цьому проєкті
Summary (тема)	Що, де і коли сталося — одним реченням
Description (опис)	Докладний опис проблеми та передумови
Steps To Reproduce	Кроки відтворення: команда запуску, YAML-сценарій, значення seed, номер кроку симуляції, на якому проявляється дефект
Actual / Expected Result	Фактичний і очікуваний результат з посиланням на критерій приймання
Attachments (вкладення)	Журнал виконання, файл сценарію, знімок екрана або CSV з метриками
Severity (серйозність)	S1–S7 за додатком Б; встановлює тестувальник
Priority (пріоритет)	High, Medium, Low; встановлює Product Owner
Status (статус)	Поточний стан за життєвим циклом (п. 6.12)
Оточення	Операційна система, версія Python, тег або хеш коміту, версії Mesa та Solara; для інтерфейсу — браузер

Значення seed і файл сценарію обов'язкові: без них дефект імітаційної моделі неможливо відтворити, і його буде відхилено.

Додаток Б. Шкала серйозності дефектів

Рівень	Назва	Приклад для проєкту
S1	Блокер (Blocker)	Модель аварійно завершується на першому кроці; жоден сценарій не запускається
S2	Критична (Critical)	Одна ціль закріплена за двома дронами; дрон проходить крізь перешкоду; порушено safe distance
S3	Значна (Major)	Дрон не повертається на базу, коли заряд дорівнює порогу; обійти можна зміною порога
S4	Незначна (Minor)	Графік знайдених цілей оновлюється лише після паузи
S5	Незручність (Tweak)	Слайдер кількості дронів має надто великий крок
S6	Текст (Text)	Одрук у повідомленні про помилку конфігурації
S7	Тривіальна (Trivial)	Попередження сторонньої бібліотеки в консолі, що не впливає на результат